Digital Twin Capture & Processing Stack: Design Scope

Date	2026-10-18
Status	Draft v1, ready for on-device bring-up
Hardware	Jetson AGX Orin 64 GB (JetPack 6.x), Livox Mid-360 **or** RoboSense Airy (96-beam), ZED 2i
Research basis	`research/00–07` (repo facts verified against upstream source on 2026-10-18)

---

1. Intent

Build a **construction-site digital twin** (usable for any place) from a sensor rig we own. The first deliverable is a **repeatable capture → map → change-over-time** loop that runs on the Orin, plus an offline pipeline that turns captures into a measurable point cloud, a 4D change map, and a photoreal (Gaussian splat) view.

The long-term product direction is autonomous, frequent capture by a Flo Mobility robot with a fused live layer. That direction shapes the design: the robot needs to be able to reuse everything here. This document scopes **v1**, which can be handheld or robot-mounted.

Goals (v1)

1. **One stack, two LiDARs.** Switching between Mid-360 and Airy is a single launch argument. Nothing downstream of the drivers knows which LiDAR is attached.
2. **Live on the Orin:** sensor drivers, PTP time sync, LiDAR-inertial odometry (FAST-LIO2), health monitoring, recording.
3. **Offline:** globally consistent map (GLIM), clean point-cloud export (LAZ/COPC), session-to-session alignment, change detection, COLMAP export for Gaussian splatting.
4. Reproducible setup: scripts that take a fresh JetPack 6 Orin to a working stack.

Non-goals (v1)

- Survey-grade QA (LOA30/40). Expected accuracy is 2–5 cm site-wide and 1–2 cm locally.
- BIM/IFC registration, schedule linking, live worker/equipment tracking, web product UI. These are later milestones (§12).
- Running Gaussian-splat training or SAM-class models on the Orin. Those go on an x86 RTX machine.

- Real-world georeferencing. There is no RTK yet, so the site-local frame is anchored by markers.

Anti-goals (failure even if it "works")

- LiDAR-specific logic leaking into SLAM, recording or the pipeline. That would mean two stacks to maintain.
- Unsynchronised timestamps silently corrupting maps and colourisation. Sync must be **measured and reported**, not assumed.
- Forking or patching GPL upstream code when a thin adapter will do.
- Captures that can't be re-processed later. Raw data plus metadata must always be kept.

---

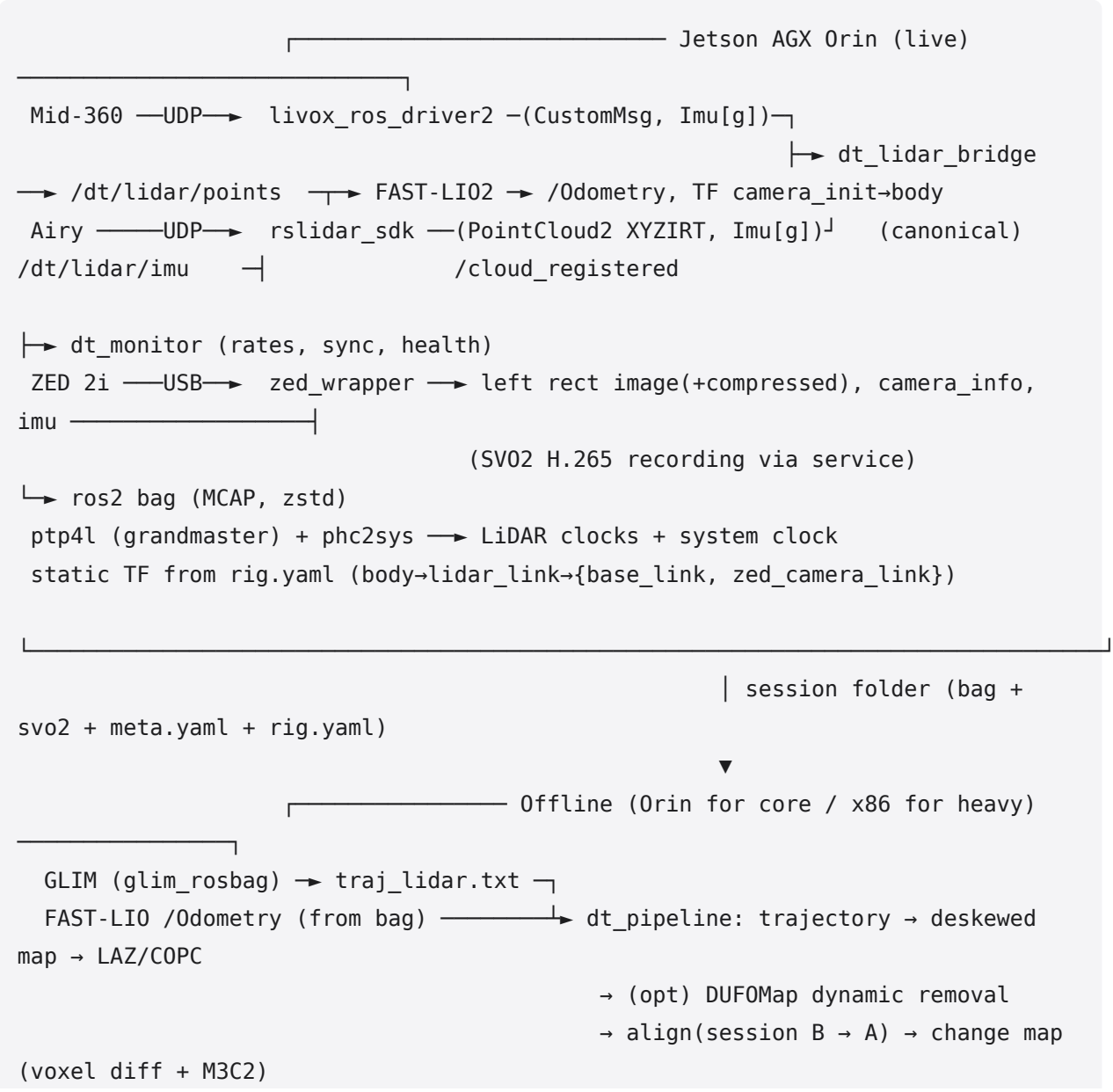
2. Constraints (verified facts that drive the design)

Fact	Consequence
JetPack 6 = Ubuntu 22.04	**ROS 2 Humble**. Native install, no Docker for the core stack.
FAST-LIO `lidar_type 4` (MID360 PointCloud2) ignores per-point time	Don't use it. Feed FAST-LIO a canonical Velodyne-style cloud (`lidar_type 2`) for **both** LiDARs.
FAST-LIO uses `points.back().curvature` as scan end time	Canonical points must be **sorted by time**.
GLIM accepts PointCloud2 only (time field `t`/`time`/`timestamp`)	The Mid-360 CustomMsg needs conversion. The canonical cloud serves both FAST-LIO and GLIM.
Both LiDAR IMUs report acceleration **in g**	The bridge scales to m/s². Downstream uses `acc_scale 1.0`.
Airy IMU shares frame_id with the cloud but is physically rotated, `R=[0,-1,0; -1,0,0; 0,0,-1]`	The bridge rotates the IMU into LiDAR axes. Downstream extrinsic rotation is then always identity.
Livox `header.stamp` = time since power-on unless PTP is locked	PTP is mandatory. The monitor detects "since-boot" stamps.
ZED is USB with host timestamps only	Camera↔LiDAR sync is software. The offset is measured and reported.
rslidar_sdk: `set(POINT_TYPE XYZI)` (plain set) and `ENABLE_IMU_DATA_PARSE OFF`	The setup script patches its CMakeLists to XYZIRT + IMU ON.
livox_ros_driver2 `build.sh` wipes the workspace	Don't use it. Copy `package_ROS2.xml` and `launch_ROS2`, then build with `-DROS_EDITION=ROS2 -DDISTRO_ROS=humble`.

Fact	Consequence
FAST-LIO hard-codes frames `camera_init` / `body`	Our TF tree adopts `body` as the IMU frame name.
GLIM has an arm64 Humble PPA (`ros-humble-glim-ros-cuda12.6`), tested on Orin JP 6.1	Install from apt.
direct_visual_lidar_calibration Docker is amd64-only	Calibration runs on an x86 PC from copied bags.
`dufomap` and `py4dgeo` have no aarch64 wheels	The offline pipeline treats them as **optional** imports. It runs on Orin (core) or x86 (full).

---

3. System overview



```
                                                  → COLMAP export (images + poses +
LiDAR points) → gsplat (x86)

└──────────────────────────────────────────────────────────────────────────────────┘
```

---

4. The canonical sensor contract

Everything downstream of `dt_lidar_bridge` depends **only** on this contract.

4.1 `/dt/lidar/points`: `sensor_msgs/PointCloud2`

Field	Type	Meaning
x, y, z	float32	metres, in `lidar_link`
intensity	float32	Livox reflectivity (0–255) or RoboSense intensity
time	float32	**seconds relative to `header.stamp`**, ≥ 0, points sorted ascending
ring	uint16	laser/line index (Mid-360 `line` 0–3; Airy ring 0–95)

- `header.stamp` is the absolute time of the earliest point (PTP-disciplined). `header.frame_id = lidar_link`.
- NaN and out-of-range points (`min_range`/`max_range`) are dropped. `height=1`, `is_dense=true`.
- Livox points are filtered by tag the same way FAST-LIO's Avia handler does: keep `(tag & 0x30) ∈ {0x00, 0x10}`. This is configurable.

Consumers and how they read it:

- **FAST-LIO** `lidar_type: 2` reads `velodyne_ros::Point {x,y,z,intensity,time,ring}` with `timestamp_unit: 0`.
- **GLIM** finds `time`, recognises it as relative (max < 1), and uses `intensity`.
- **KISS-ICP** uses `time`.

4.2 `/dt/lidar/imu`: `sensor_msgs/Imu`

- `linear_acceleration` in **m/s²**, `angular_velocity` in rad/s.
- Axes rotated to be **parallel to `lidar_link`**.
- `frame_id = body`. `orientation_covariance[0] = -1`, meaning no orientation.
- The IMU origin is offset from the LiDAR origin by `T_lidar_imu.translation` (from the profile).

4.3 Camera topics (ZED wrapper v5 names, used as-is)

- `/zed/zed_node/left/color/rect/image` (+ `/compressed`) and `.../left/color/rect/camera_info`
- `/zed/zed_node/imu/data`

- Depth `/zed/zed_node/depth/depth_registered`. Only needed for nvblox (M3).

4.4 TF tree

```
map ──(static I | GLIM)──► odom ──(static I)──► camera_init ──(FAST-LIO)──► body (=IMU)

└─(static)─► lidar_link ─┬─(static)─► base_link

└─(static)─► zed_camera_link ─(ZED URDF)─► zed_* optical frames
```

- All static transforms come from the profile's `rig.yaml`, so there is a single source of truth.
- `body → lidar_link` is the inverse of the IMU position in the LiDAR frame, with rotation identity after the bridge.
- `lidar_link → base_link` is the inverse of the mount, so the robot/tripod base hangs off the sensor.

---

5. Profiles: one argument switches LiDARs

`ros2_ws/src/dt_bringup/config/profiles/<profile>/` with `profile ∈ {mid360, airy}`:

File	Content
`rig.yaml`	`T_base_lidar` (mount), `T_lidar_imu` (IMU position in LiDAR frame, after bridge rotation), `T_lidar_camera` (LiDAR → `zed_camera_link`), plus metadata
`bridge.yaml`	Input type, input topics, IMU rotation matrix, acc scale, range limits
`fast_lio.yaml`	FAST-LIO params on canonical topics (`lidar_type 2`, `scan_line`, `extrinsic_T`)
`glim/`	Overrides merged into GLIM's stock config by `dt_tools glim-config`
driver config	`MID360_config.json` (Livox) or `rslidar.yaml` (RoboSense)

Launch: `ros2 launch dt_bringup capture.launch.py profile:=airy camera:=true slam:=fastlio record:=false`

Profile values

	mid360	airy
IMU rotation (bridge)	I	`[0,-1,0, -1,0,0, 0,0,-1]` (rslidar_sdk #203) **[verify on device]**

	mid360	airy
`T_lidar_imu` translation	`[0.011, 0.02329, -0.04412]` (Livox spec, = −FAST-LIO `extrinsic_T`)	`[0, 0, 0]` **[verify]**
FAST-LIO `extrinsic_T` (LiDAR in IMU frame)	`[-0.011, -0.02329, 0.04412]`	`[0, 0, 0]`
FAST-LIO `scan_line` / `point_filter_num`	4 / 3	96 / 6
Default IPs	LiDAR `192.168.1.1XX` (serial), host `192.168.1.5`	LiDAR `192.168.1.200`, host `192.168.1.102`

---

6. Components (what we write vs. what we clone)

6.1 Cloned upstream (pinned in `ros2_ws/dt.repos`, imported with `vcs import --recursive`)

Repo	Ref	Role	License
Livox-SDK/Livox-SDK2	master	SDK (built to /usr/local by script)	MIT
Livox-SDK/livox_ros_driver2	master	Mid-360 driver	MIT
RoboSense-LiDAR/rslidar_sdk	v1.5.20 (+ submodule)	Airy driver	BSD-3
RoboSense-LiDAR/rslidar_msg	master	msgs for rslidar_sdk	BSD-3
hku-mars/FAST_LIO	ROS2 (+ ikd-Tree)	live LIO	GPL-2.0 (unmodified, run as separate process)
stereolabs/zed-ros2-wrapper	tag matching installed SDK (v5.5.0 for SDK 5.5)	ZED driver	Apache-2.0
GLIM	apt PPA `ros-humble-glim-ros-cuda12.x`	global mapping (offline/online)	MIT

After the first successful build, `vcs export --exact > dt.lock.repos` freezes the commits.

6.2 Ours

Package	Lang	Responsibility
`dt_lidar_bridge`	C++ (rclcpp component + node)	`PointsBridge` (livox CustomMsg / generic XYZIRT → canonical), `ImuBridge` (scale + rotate + reframe)
`dt_bringup`	launch / config	profiles, `sensors.launch.py`, `slam.launch.py`, `capture.launch.py`, rig static TFs, RViz config
`dt_tools`	Python (ament_python)	`dt_monitor` (rates, stamp sanity, cross-sensor offset), `session` (record start/stop + SVO + metadata), `glim-config` generator, `apply-camera-calib`
`pipeline/dt_pipeline`	Python (pip, ROS-free)	bag reading, trajectory, map building, cleaning, alignment, change detection, COLMAP export
`scripts/jetson/*`	bash	system check, deps, SDKs, network, PTP, build

---

7. Time synchronisation

1. **PTP grandmaster on the Orin:**
   - Run `ptp4l` on the LiDAR NIC: hardware timestamping, UDPv4, E2E, domain 0, `tx_timestamp_timeout 20`.
   - Run `phc2sys` to keep the NIC clock (PHC) and `CLOCK_REALTIME` aligned.
   - Both run as systemd services installed by `scripts/jetson/06_setup_ptp.sh`.
2. **Mid-360:** acts as a PTP slave automatically.
3. **Airy:** set the time source to **PTP-E2E (L4)** in its web UI (one-time manual step, documented), and keep `use_lidar_clock: true`.
4. **ZED:** `sdk_timestamp_clock: SYSTEM_CLOCK`. This follows `CLOCK_REALTIME`, which phc2sys keeps aligned with PTP.
5. **`dt_monitor` reports for every stream:**
   - rate
   - `now − header.stamp` (latency)
   - a "since-boot" detector (stamp < year 2020)
   - LiDAR↔IMU and LiDAR↔camera stamp offsets It raises a WARN if latency is > 100 ms or < −10 ms, or if the PTP state is unknown.
6. **Residual camera offset:** a known residual LiDAR↔camera offset is stored in `rig.yaml: camera_time_offset` and applied by the pipeline.

---

8. Recording and sessions

```
~/dt_data/sessions/<site>/<YYYYmmdd_HHMMSS>_<profile>/
  bag/            ros2 bag, MCAP, zstd_fast preset
  zed.svo2        full-quality stereo (H.265 NVENC) via
/zed/zed_node/start_svo_rec
  meta.yaml       profile, operator note, git describe, topics, start/stop, ptp
status snapshot
  rig.yaml        copy of the profile rig at capture time
```

Default topic set:

- canonical points + IMU
- raw driver topics (livox CustomMsg+IMU, or rslidar points+IMU)
- `/zed/zed_node/left/color/rect/image/compressed` + camera_info
- `/zed/zed_node/imu/data`
- `/Odometry`, `/tf`, `/tf_static`

Lean mode (`--lean`) drops the raw driver topics.

Not recorded: raw ZED images, depth, and the ZED point cloud. The SVO2 holds the full-quality images.

Performance setup:

- `sudo nvpmodel -m 0 && sudo jetson_clocks`
- Record to NVMe.
- `--max-cache-size 1 GiB`.

---

9. SLAM

Online: FAST-LIO2 (live odometry and TF)

- Separate process (GPL boundary), on canonical topics, `lidar_type: 2`, `timestamp_unit: 0`, `extrinsic_est_en: false`.
- Map auto-save: off by default. The map is built offline.
- **Initialisation:** the sensor must be **static for ~2 s** at start-up (documented in the operator checklist).

Offline: GLIM (global consistency, multi-session)

1. `dt_tools glim-config --profile X --out <dir>` copies GLIM's stock config and patches:
    - topics → canonical
    - `acc_scale 1.0`
    - `T_lidar_imu` from `rig.yaml`

    - headless extension modules
    - GPU/CPU choice
2. `ros2 run glim_ros glim_rosbag <bag> --ros-args -p config_path:=<dir>` produces a dump.
3. `dt_pipeline` reads `traj_lidar.txt` (TUM, `T_world_lidar`).

Fallback: `dt_pipeline` can use the FAST-LIO `/Odometry` recorded in the bag. Odometry gives `T_odom_body`, which is converted to `T_world_lidar` using `T_lidar_imu`.

Later (deferred)

- Loop closure across sessions with GLIM multi-session, or our own pose graph.
- Airy-specific Point-LIO for aggressive handheld motion.

---

10. Offline pipeline (`dt_pipeline`, CLI `dtp`)

ROS-free. Runs on the Orin (core) or x86 (full). Each step writes into `<session>/products/`.

Command	Input → Output	Algorithm
`dtp info <session>`	bag → topic/rate/duration summary	rosbags
`dtp traj <session> --source glim|fastlio`	GLIM dump or `/Odometry` → `products/traj_lidar.tum`	TUM I/O, extrinsic conversion
`dtp map <session>`	canonical scans + trajectory → `products/map.laz` (+ `.copc.laz` if PDAL)	per-point deskew by interpolating pose at `stamp+time`, voxel downsample, range/intensity filters
`dtp clean <session>`	scans + traj → `products/map_static.laz`, `map_dynamic.laz`	DUFOMap (optional dep), fallback: voxel visibility count
`dtp align <ref> <session>`	two maps → `products/T_ref_session.txt`	Open3D FPFH+RANSAC (coarse, skippable when markers or known init) → small_gicp GICP (fine)
`dtp diff <ref> <session>`	aligned maps → `products/diff/{added,removed}.laz`, `m3c2.laz`, `summary.json`	voxel occupancy diff (5 cm), optional py4dgeo M3C2 with level of detection
`dtp colmap <session>`	images (bag) + traj + calib + map → `products/colmap/{images,sparse/0}`	`T_world_cam(t) = T_world_lidar(t + Δt) ·`

Command	Input → Output	Algorithm
		`T_lidar_cam_optical`, PINHOLE from camera_info, `points3D` = coloured voxelised map

Pose interpolation: SLERP on rotation plus linear translation over the trajectory. Points outside the trajectory's time span are dropped.

---

11. Calibration workflow

1. **Intrinsics:** use the ZED factory calibration (rectified images, so zero distortion).
2. **LiDAR↔IMU:** use the profile values.
   - Validate with GLIM `libimu_validator.so` on a capture.
   - Validate that FAST-LIO doesn't diverge when you rotate in place.
3. **LiDAR↔camera:**
   1. `dt_tools record-calib` records 6 short static bags (Mid-360: hold still for 15 s; Airy: slow up/down sweep for 10 s).
   2. Copy the bags to an x86 PC and run `direct_visual_lidar_calibration` (Docker `humble`): `preprocess -av/-adv`, then `initial_guess_manual` (no SuperGlue, because of its licence), then `calibrate`. This produces `calib.json` (`T_lidar_camera_optical`).
   3. On the Orin with the ZED running, `dt_tools apply-camera-calib calib.json --profile X` looks up `zed_camera_link→<optical frame>` from TF and writes `T_lidar_camera` (to `zed_camera_link`) and `T_lidar_camera_optical` into `rig.yaml`.
4. **Verify:** `dtp overlay <session>` projects LiDAR points onto images. It is part of M2.

---

12. Milestones

M	Scope	Exit criterion
M0 Bring-up (tomorrow)	setup scripts, both drivers, bridge, ZED, monitor	`dt_monitor` green: points 10 Hz, IMU ≥ 100 Hz (Mid-360 200 Hz), image 15/30 Hz, stamps PTP-sane
M1 Live LIO + record	FAST-LIO on canonical topics for both profiles; session recorder	2-minute loop walk returns to start with < 0.2 m drift; session folder complete

M	Scope	Exit criterion
M2 Offline map + calib	GLIM offline, `dtp traj/map`, camera calibration, overlay check	Clean `map.laz` viewed in CloudCompare/Potree; overlay visually aligned
M3 4D + live mapping	`dtp align/diff`, DUFOMap; nvblox (Isaac ROS 3.2) live mesh	Change map between two captures of the same area highlights a moved object
M4 Visual twin	`dtp colmap` → gsplat on x86 (depth-supervised, pose-opt), SOG/HTML viewer	Walkable splat of a test area
M5 Semantics	RF-DETR (people/PPE/equipment) TensorRT on Orin; SAM 3 offline labelling; 3D lifting	People layer separated from the static map
M6	RTK georeferencing, BIM (IfcOpenShell) deviation, robot autonomy, live layer	n/a

---

13. Repository layout

```
digital-twin/
  docs/design/            this document
  research/               landscape, hardware, algorithms, verified repo facts
  ros2_ws/
    dt.repos              upstream repos (vcstool)
    src/
      dt_lidar_bridge/    C++ canonical adapter
      dt_bringup/         launch + profiles + rviz
      dt_tools/           monitor, session recorder, config generators
      external/           vcs-imported upstream (git-ignored)
  pipeline/               dt_pipeline python package (+ tests)
  scripts/jetson/         00_check … 07_build
  scripts/capture/        convenience wrappers
```

---

14. Risks and open items (verify on device)

#	Item	Mitigation
1	Airy IMU rotation and translation	`dt_monitor --imu-check`: at rest, canonical accel must be ≈ (0, 0, +9.81) when the LiDAR is upright. Profile values are editable.

#	Item	Mitigation
2	Airy publishing < 10 Hz on Jetson (#258)	CycloneDDS with larger socket buffers (`scripts/jetson/05_network.sh` sets `rmem_max`); bridge as component in the same container as the driver when possible
3	Orin MGBE PTP stability	`tx_timestamp_timeout 20`; fall back to software timestamping; monitor flags it
4	UTC/TAI 37 s offset	Monitor detects a constant ~37 s offset and prints the `-0` fix
5	GLIM CUDA package vs JetPack CUDA version	Script picks `cuda12.6` for L4T 36.4+, `cuda12.2` for 36.3. CPU config fallback.
6	ZED SDK / wrapper version match	Script detects L4T, installs the matching SDK, checks out the matching wrapper tag
7	Livox and Airy on the same subnet	NIC gets both host IPs (`.5` and `.102`). Only one LiDAR is active per profile.

15. Decisions made by Claude (please review)

- The canonical-bridge architecture, and running FAST-LIO via `lidar_type 2` for both LiDARs instead of the native Livox path or the KiiiLin Airy fork.
- `body` as the IMU frame name, to match FAST-LIO's hard-coded TF.
- Native install (no Docker) for the core stack; calibration on x86.
- Recording compressed images plus SVO2, and no raw images.
- The offline pipeline is ROS-free Python, with heavy optional deps.